

C64 / Guía rápida

6510 + Kick Assembler 5.25

Una guía de consulta en español para programar la Commodore 64 en assembler. Entradas breves, agrupadas por tema y ordenadas alfabéticamente cuando corresponde.

BUSCAR RÁPIDO

- 01 / Registros y direccionamiento / página 2
- 02 / Mnemónicos oficiales: adc - tya (pp. 3-5) / página 3
- 03 / Opcodes no oficiales / página 6
- 04 / Operadores de Kick Assembler / página 7
- 05 / Sintaxis, números y texto / página 8
- 06 / Directivas de Kick Assembler (pp. 9-11) / página 9
- 07 / Preprocesador y utilidades / página 12
- 08 / Las 39 rutinas KERNAL (pp. 13-15) / página 13
- 09 / ROM: uso práctico y entradas internas / página 16
- 10 / Memoria y video: recordatorio / página 17
- 11 / Fuentes y alcance / página 18

TRES CAPAS DIFERENTES

Capa	Dónde actúa
Instrucción: lda, sta, bne	La ejecuta el 6510 mientras corre tu programa.
Directiva: .byte, .fill, .for	La procesa Kick Assembler en Windows para construir el programa.
Rutina: CHROUT, GETIN	Es código ya presente en ROM; tu programa lo llama con parámetros.

Alcance: los 56 mnemónicos oficiales del 6510, los no oficiales listados por Kick Assembler 5.25, sus operadores documentados, directivas y preprocesador; las 39 entradas públicas KERNAL y una selección explícita de entradas internas de ROM. No es un inventario de todos los puntos internos de BASIC/KERNAL.

Las formas de sintaxis son referencias aisladas. No incluye una implementación del juego ni modifica tu archivo main.asm.

Consulta con Ctrl+F, el índice enlazado o los marcadores del lector PDF. Edición: 6 de octubre de 2026.

01 / Registros y direccionamiento

Qué significa cada operando. Los modos disponibles dependen de la instrucción.

Registro / flag	Función
A / X / Y	Registros de 8 bits. A: acumulador; X e Y: índices y datos. Sus valores van de 0 a 255.
PC / SP	PC: dirección de la próxima instrucción, 16 bits. SP: índice de la pila en \$0100-\$01FF, 8 bits.
P	Registro de estado: N V - B D I Z C. B describe el origen de la copia apilada; no es un flag persistente normal.
N / Z	N copia el bit 7 del resultado. Z=1 cuando el resultado es cero; Z=0 en caso contrario.
C / V	C: acarreo; en resta, 1 significa sin préstamo. V: desbordamiento con signo, distinto de C.
D / I	D=1: aritmética decimal BCD en adc/sbc. I=1: bloquea IRQ enmascarables, no NMI.

MODOS DE DIRECCIONAMIENTO

Referencia	Descripción
imp / acc	Sin operando. Operación implícita; asl, lsr, rol y ror sin operando actúan sobre A.
imm: #valor	Valor literal de 8 bits. No lee esa dirección de memoria.
zp: \$nn	Dirección de página cero: \$0000-\$00FF. Instrucción normalmente de 2 bytes.
zp,X / zp,Y	Base de página cero más X o Y; la suma vuelve a \$00 al superar \$FF.
abs: \$nnnn	Dirección de 16 bits. Instrucción normalmente de 3 bytes.
abs,X / abs,Y	Dirección de 16 bits más X o Y. Puede cruzar páginas de 256 bytes.
(zp,X)	Primero suma X a zp; luego lee allí un puntero de 16 bits en página cero.
(zp),Y	Primero lee el puntero de 16 bits en zp; después suma Y a la dirección obtenida.
ind: (addr)	jmp indirecto: toma de memoria la dirección de destino.
rel: etiqueta	Salto condicionales: desplazamiento -128 a +127 desde la instrucción siguiente.

Los punteros guardan primero el byte bajo y después el alto (little endian). En el 6510, jmp indirecto con puntero terminado en \$FF toma el byte alto del inicio de esa misma página: evitá ese límite.

Un mnemónico es el nombre, por ejemplo lda. Un opcode es el byte que identifica la instrucción y su modo; lda inmediato usa \$A9 y lda absoluto usa \$AD.

02 / Mnemónicos: adc - cpx

Los 56 oficiales del 6502/6510. Escríbelos en minúsculas en Kick Assembler.

Nombre	Para qué sirve	Modos	Flags
adc	Suma A + operando + C y deja el byte resultante en A. D selecciona binario o BCD.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z C V
and	AND bit a bit entre A y el operando; deja el resultado en A. Sirve para conservar o borrar bits.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z
asl	Desplaza un bit a la izquierda; entra 0 por bit 0 y el antiguo bit 7 pasa a C.	acc, zp, zpX, abs, absX	N Z C
bcc	Salta si C=0. Tras una comparación sin signo: menor.	rel	-
bcs	Salta si C=1. Tras una comparación sin signo: mayor o igual.	rel	-
beq	Salta si Z=1. No compara: consulta el flag que dejó una instrucción anterior.	rel	-
bit	Z depende de A AND memoria; N y V copian los bits 7 y 6 de memoria. No cambia A.	zp, abs	N Z V
bmi	Salta si N=1: bit 7 del resultado activado.	rel	-
bne	Salta si Z=0: resultado anterior distinto de cero.	rel	-
bpl	Salta si N=0. Incluye el resultado cero.	rel	-
brk	Interrupción por software. Apila PC+2 y estado; usa el vector IRQ/BRK. No equivale a terminar un programa.	imp	I
bvc	Salta si V=0: no hay desbordamiento con signo indicado.	rel	-
bvs	Salta si V=1: hay desbordamiento con signo indicado.	rel	-
clc	Pone C=0. Se usa antes de una suma sin acarreo de entrada.	imp	C
cld	Pone D=0: adc y sbc trabajan en binario.	imp	D
cli	Pone I=0: permite IRQ enmascarables.	imp	I
clv	Pone V=0.	imp	V
cmp	Compara A con el operando mediante una resta sin guardar el resultado. A se conserva.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z C
cpx	Compara X con el operando; no modifica X.	imm, zp, abs	N Z C

zpX = zp,X; zpY = zp,Y; absX = abs,X; absY = abs,Y; izX = (zp,X); izY = (zp),Y. Un guion indica que no modifica flags.

Las descripciones de N/Z/V en aritmética suponen D=0. En el 6510 NMOS, los flags de adc/sbc en modo decimal tienen particularidades: no los interpretes como si describieran siempre el resultado BCD corregido.

02 / Mnemónicos: cpy - pla

Los 56 oficiales del 6502/6510. Escribilos en minúsculas en Kick Assembler.

Nombre	Para qué sirve	Modos	Flags
cpy	Compara Y con el operando; no modifica Y.	imm, zp, abs	N Z C
dec	Resta 1 al byte de memoria. \$00 pasa a \$FF. No modifica C.	zp, zpX, abs, absX	N Z
dex	Resta 1 a X; 0 pasa a 255.	imp	N Z
dey	Resta 1 a Y; 0 pasa a 255.	imp	N Z
eor	XOR bit a bit entre A y el operando. Los bits con máscara 1 se invierten.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z
inc	Suma 1 al byte de memoria. \$FF pasa a \$00. No modifica C.	zp, zpX, abs, absX	N Z
inx	Suma 1 a X; 255 pasa a 0 y activa Z.	imp	N Z
iny	Suma 1 a Y; 255 pasa a 0 y activa Z.	imp	N Z
jmp	Transfiere el control al destino, sin guardar retorno.	abs, ind	-
jsr	Llama a una subrutina: apila el retorno y salta. rts permite volver.	abs	-
lda	Carga el operando en A. Con # toma un literal; sin # lee memoria.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z
ldx	Carga el operando en X. El modo indexado de memoria usa Y.	imm, zp, zpY, abs, absY	N Z
ldy	Carga el operando en Y. El modo indexado de memoria usa X.	imm, zp, zpX, abs, absX	N Z
lsr	Desplaza un bit a la derecha; entra 0 por bit 7 y el antiguo bit 0 pasa a C. N queda en 0.	acc, zp, zpX, abs, absX	N Z C
nop	No realiza una operación útil; el opcode oficial \$EA ocupa 1 byte y tarda 2 ciclos.	imp	-
ora	OR bit a bit entre A y el operando. Una máscara con bits 1 permite activarlos.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z
pha	Apila A. A se conserva; SP disminuye.	imp	-
php	Apila una copia del estado con B y bit 5 en 1.	imp	-
pla	Desapila un byte en A y actualiza N y Z.	imp	N Z

zpX = zp,X; zpY = zp,Y; absX = abs,X; absY = abs,Y; izX = (zp,X); izY = (zp),Y. Un guion indica que no modifica flags.

Las descripciones de N/Z/V en aritmética suponen D=0. En el 6510 NMOS, los flags de adc/sbc en modo decimal tienen particularidades: no los interpretes como si describieran siempre el resultado BCD corregido.

02 / Mnemónicos: plp - tya

Los 56 oficiales del 6502/6510. Escribilos en minúsculas en Kick Assembler.

Nombre	Para qué sirve	Modos	Flags
plp	Restaura los flags desde la pila. B y bit 5 no son flags persistentes restaurables.	imp	N V D I Z C
rol	Rota a la izquierda pasando por C: entra el C anterior por bit 0; sale bit 7 hacia C.	acc, zp, zpX, abs, absX	N Z C
ror	Rota a la derecha pasando por C: entra el C anterior por bit 7; sale bit 0 hacia C.	acc, zp, zpX, abs, absX	N Z C
rti	Retorna de una interrupción: restaura estado y PC desde la pila.	imp	N V D I Z C
rts	Retorna de una subrutina: recupera la dirección apilada y suma 1. No restaura los flags.	imp	-
sbc	Calcula A - operando - (1-C). Para restar sin préstamo previo se prepara C=1.	imm, zp, zpX, abs, absX, absY, izX, izY	N Z C V
sec	Pone C=1.	imp	C
sed	Pone D=1: adc y sbc operan en decimal codificado en binario (BCD).	imp	D
sei	Pone I=1: bloquea IRQ enmascarables; NMI sigue habilitada.	imp	I
sta	Guarda A en memoria. No cambia A ni los flags.	zp, zpX, abs, absX, absY, izX, izY	-
stx	Guarda X en memoria. No cambia X ni los flags.	zp, zpY, abs	-
sty	Guarda Y en memoria. No cambia Y ni los flags.	zp, zpX, abs	-
tax	Copia A en X. A se conserva.	imp	N Z
tay	Copia A en Y. A se conserva.	imp	N Z
tsx	Copia SP en X.	imp	N Z
txa	Copia X en A. X se conserva.	imp	N Z
txs	Copia X en SP. No modifica los flags.	imp	-
tya	Copia Y en A. Y se conserva.	imp	N Z

zpX = zp,X; zpY = zp,Y; absX = abs,X; absY = abs,Y; izX = (zp,X); izY = (zp),Y. Un guion indica que no modifica flags.

Las descripciones de N/Z/V en aritmética suponen D=0. En el 6510 NMOS, los flags de adc/sbc en modo decimal tienen particularidades: no los interpretes como si describieran siempre el resultado BCD corregido.

03 / Opcodes no oficiales

Reconocidos por Kick Assembler con `.cpu _6502`. Referencia avanzada; no necesarios para aprender.

Nombre / alias	Efecto resumido
<code>ahx / sha</code>	Guarda una combinación de A, X y el byte alto de la dirección +1. Inestable en ciertos casos de direccionamiento.
<code>alr / asr</code>	Combina AND inmediato con desplazamiento lógico a la derecha de A.
<code>anc</code>	AND inmediato; además copia el bit 7 del resultado a C. Opcode \$0B.
<code>anc2</code>	Variante de anc con opcode \$2B.
<code>arr</code>	Combina AND inmediato y rotación a la derecha de A; C, V y el modo decimal tienen reglas especiales.
<code>axs / sbx</code>	Calcula (A AND X) menos el inmediato y lo deja en X; actualiza N, Z y C.
<code>dcp / dcm</code>	Decrementa memoria y luego la compara con A.
<code>isc / ins / isb</code>	Incrementa memoria y luego la resta de A mediante sbc.
<code>las / lae / lds</code>	Combina memoria AND SP y copia el resultado en A, X y SP.
<code>lax / lxa</code>	Carga A y X a la vez. La forma inmediata \$AB tiene comportamiento dependiente del chip.
<code>nop (variantes)</code>	Versiones que consumen operandos, bytes y ciclos adicionales; algunas leen memoria.
<code>r1a</code>	Rota memoria a la izquierda y hace AND con A.
<code>rra</code>	Rota memoria a la derecha y suma el resultado a A con acarreo.
<code>sax</code>	Guarda A AND X en memoria, sin cambiar A ni X.
<code>sbc2</code>	Variante inmediata de sbc con opcode \$EB.
<code>shx</code>	Almacenamiento enmascarado con X y el byte alto de dirección; comportamiento especial al cruzar página.
<code>shy</code>	Almacenamiento enmascarado con Y y el byte alto de dirección; comportamiento especial al cruzar página.
<code>slo</code>	Desplaza memoria a la izquierda y hace OR con A.
<code>sre</code>	Desplaza memoria a la derecha y hace XOR con A.
<code>tas / shs</code>	Copia A AND X a SP y realiza un almacenamiento enmascarado dependiente de la dirección.
<code>xaa / ane</code>	Operación combinada sobre A, X e inmediato; resultado inestable y dependiente del chip.

El 6510 también tiene bytes KIL/JAM que bloquean la CPU hasta un reset. No aparecen como mnemónico en la tabla de Kick Assembler 5.25.

No confundir con instrucciones del 65C02 o C64 DTV: `bra`, `stz`, `phx`, `phy`, `plx`, `ply`, `trb`, `tsb` y las extensiones de esas CPU no pertenecen a la C64 clásica.

Para limitarse al conjunto oficial se puede seleccionar `.cpu _6502NoIllegals`. El conjunto predeterminado de Kick Assembler es `_6502`, que admite también los no oficiales.

04 / Operadores de Kick Assembler

Se evalúan en Windows durante el ensamblado. No ejecutan cálculos en el 6510.

Operador	Significado / uso
<code>-x</code>	Negación aritmética: cambia el signo.
<code>+ (suma), - (resta)</code>	Suma y resta. + también concatena cadenas.
<code>* y /</code>	Multiplicación y división. Los números del lenguaje de script pueden ser fraccionarios.
<code><valor / >valor</code>	Operadores unarios: byte bajo y byte alto. Para \$1234 producen \$34 y \$12.
<code>a << n / a >> n</code>	Desplaza bits a izquierda o derecha durante el ensamblado.
<code>a & b</code>	AND bit a bit. Útil para máscaras.
<code>a b</code>	OR bit a bit. Combina bits.
<code>a ^ b</code>	XOR bit a bit. Invierte las posiciones seleccionadas.
<code>~a</code>	Complemento bit a bit. Para restringir a un byte, aplicar máscara \$FF.
<code>a == b / a != b</code>	Igualdad y desigualdad; producen un booleano.
<code>a < b / a > b</code>	Comparaciones menor y mayor. Con dos operandos no extraen bytes.
<code>a <= b / a >= b</code>	Comparaciones menor o igual y mayor o igual.
<code>!a</code>	Negación lógica de un booleano.
<code>a && b / a b</code>	AND / OR lógicos con evaluación de cortocircuito.
<code>cond ? a : b</code>	Expresión condicional: elige a si cond es true; b si es false.
<code>=</code>	Asignación en el lenguaje de script. No compara ni almacena en RAM de la C64.
<code>+= / -= / *= / /=</code>	Actualiza una variable de ensamblado combinando operación y asignación.
<code>x++ / x--</code>	Incremento / decremento posfijo; la expresión devuelve el valor anterior.
<code>(...) / [...]</code>	Agrupar expresiones. En operandos de CPU, (...) puede significar direccionamiento indirecto; [...] evita esa ambigüedad.
<code>*</code>	Contador de posición actual cuando aparece como valor. Entre dos operandos significa multiplicación.

No trasladar toda la sintaxis de PHP/Java/C sin verificar: esta tabla recoge los operadores documentados en el manual instalado. Para resto de división, el manual ofrece la función `mod(a,b)`; % se usa como prefijo binario.

Una expresión como `dirección+40` la resuelve el ensamblador. Una instrucción como `adc` realiza una suma mientras corre el programa: son momentos distintos.

05 / Sintaxis, números y texto

Convenciones que vas a consultar al escribir un archivo .asm.

Forma	Qué significa
42 / \$2A / %101010	El mismo número en decimal, hexadecimal y binario. Sin prefijo se interpreta decimal.
#valor	Operando inmediato de una instrucción: el número en sí, no el contenido de una dirección.
nombre:	Define una etiqueta en la posición actual. Referenciarla sin los dos puntos.
!nombre: / !:	Etiqueta reutilizable o anónima. !nombre+ busca la siguiente; !nombre- la anterior; !+ y !- para anónimas.
!++ / !--	Salta dos apariciones de una etiqueta anónima hacia delante / atrás; se pueden repetir los signos.
argumento:	Una etiqueta delante de un operando marca su ubicación dentro de la instrucción. Uso avanzado: código automodificable.
// comentario	Comentario hasta el fin de línea.
/* comentario */	Comentario de bloque, que puede ocupar varias líneas.
;	Separa instrucciones o partes de .for. NO inicia comentarios en Kick Assembler.
{ ... }	Agrupar directivas y crea ámbitos locales según el contexto.
espacio.nombre	Acceso a un símbolo de un namespace o ámbito etiquetado.
@nombre	Busca el símbolo en el ámbito raíz, escapando al ámbito local.
"texto" / 'A'	Cadena y carácter. Su conversión a bytes depende de .encoding.
@"texto"	Cadena con interpretación de secuencias de escape de Kick Assembler.
.abs / .a	Sufijo del mnemónico para forzar direccionamiento absoluto cuando existe.
.zp / .z	Sufijo del mnemónico para forzar página cero cuando existe.
virtual	Opción de un bloque de memoria que define posiciones sin incluir sus bytes en el archivo generado.
true / false / null	Valores booleanos y ausencia de valor en el lenguaje de script.

CODIFICACIONES PARA .encoding

Referencia	Descripción
ascii	Codificación ASCII; no es directamente la de la pantalla C64.
petSCII_upper / petSCII_mixed	PETSCII para mayúsculas/gráficos o mayúsculas/minúsculas; adecuado para salida mediante KERNAL.
screencode_upper / screencode_mixed	Códigos para la RAM de pantalla en los dos juegos de caracteres. Predeterminado del ensamblador: screencode_mixed.

.encoding transforma texto en bytes al ensamblar; no cambia la fuente ni el modo del VIC-II. Escribir .text tampoco imprime: solamente genera datos.

06 / Directivas: * = dirección - .file

Catálogo del manual instalado, incluidos alias, variantes de importación y .watch.

Directiva	Para qué sirve
<code>* = dirección</code>	Fija la posición donde se ensamblan los bytes siguientes. No ejecuta ni salta a esa dirección.
<code>.align</code>	Avanza hasta la siguiente frontera de alineación indicada; si ya está alineado, no avanza.
<code>.assert</code>	Comprueba igualdad de expresiones o de bloques ensamblados durante la construcción.
<code>.asserterror</code>	Comprueba que una expresión o bloque produzca un error esperado.
<code>.break</code>	Añade un punto de interrupción a los datos de depuración. No inserta una instrucción brk.
<code>.by</code>	Alias de .byte.
<code>.byte</code>	Emite valores de un byte en el archivo de salida.
<code>.const</code>	Declara un símbolo constante del lenguaje de ensamblado; no reserva RAM.
<code>.cpu</code>	Selecciona conjunto de instrucciones: _6502Nolllegals, _6502, dtv o _65c02.
<code>.define</code>	Ejecuta un bloque en modo de funciones para definir valores; evita registrar salida de ensamblado innecesaria.
<code>.disk</code>	Construye una imagen de disco D64 según sus parámetros y archivos.
<code>.dw</code>	Alias de .dword. Atención: NO es alias de .word.
<code>.dword</code>	Emite valores de 4 bytes en orden little endian.
<code>.encoding</code>	Selecciona la conversión de caracteres a bytes; ver la página de sintaxis.
<code>.enum</code>	Declara una secuencia de constantes, con valores automáticos o explícitos.
<code>.error</code>	Genera un error de ensamblado con un mensaje propio.
<code>.errorif</code>	Genera un error si se cumple una condición; útil para verificar límites y direcciones.
<code>.eval</code>	Evalúa una expresión de script, por ejemplo una asignación o llamada a un método.
<code>.file</code>	Genera un archivo PRG o binario a partir de los segmentos indicados.

Estas directivas se procesan al ensamblar. Las que emiten bytes preparan el archivo; no escriben por sí solas sobre una C64 que ya está ejecutando el programa.

La sintaxis completa de parámetros para segmentos, archivos y discos está en los capítulos 10 y 11 del manual de Kick Assembler.

06 / Directivas: .filemodify - .pc

Catálogo del manual instalado, incluidos alias, variantes de importación y .watch.

Directiva	Para qué sirve
<code>.filemodify</code>	Aplica un modificador a la salida del archivo fuente actual.
<code>.filenamespace</code>	Establece un namespace para el resto del archivo fuente.
<code>.fill</code>	Genera bytes repitiendo una expresión; i es el índice de repetición. También admite patrones de valores.
<code>.fillword</code>	Como .fill, pero emite palabras de 2 bytes.
<code>.for</code>	Repite directivas al ensamblar. Puede generar tablas o código repetido; no crea un bucle de CPU por sí solo.
<code>.function</code>	Define una función del lenguaje de script que calcula un resultado al ensamblar.
<code>.if</code>	Incluye o evalúa directivas según una condición de ensamblado. La alternativa se escribe else, sin punto.
<code>.import binary</code>	Incorpora bytes de un archivo binario.
<code>.import c64</code>	Incorpora un archivo omitiendo sus dos primeros bytes de dirección de carga.
<code>.import source</code>	Importa código fuente; forma antigua. Se recomienda #import.
<code>.import text</code>	Importa texto y lo convierte con la codificación activa.
<code>.importonce</code>	Evita importar de nuevo un archivo; forma antigua. Se recomienda #importonce.
<code>.label</code>	Define una etiqueta mediante una expresión; admite referencias hacia delante según el contexto.
<code>.lohifill</code>	Genera dos tablas consecutivas: bytes bajos y altos. La etiqueta asociada ofrece los miembros .lo y .hi.
<code>.macro</code>	Define un bloque expandible con parámetros. Se expande al ensamblar, sin llamada jsr en ejecución.
<code>.memblock</code>	Empieza un bloque de memoria nuevo en la posición actual; permite nombrarlo.
<code>.modify</code>	Aplica un modificador a los bytes generados dentro de un bloque.
<code>.namespace</code>	Crea un espacio de nombres para organizar símbolos y evitar colisiones.
<code>.pc</code>	Forma antigua equivalente a * = dirección.

Estas directivas se procesan al ensamblar. Las que emiten bytes preparan el archivo; no escriben por sí solas sobre una C64 que ya está ejecutando el programa.

La sintaxis completa de parámetros para segmentos, archivos y discos está en los capítulos 10 y 11 del manual de Kick Assembler.

06 / Directivas: .plugin - .zp

Catálogo del manual instalado, incluidos alias, variantes de importación y .watch.

Directiva	Para qué sirve
<code>.plugin</code>	Carga una clase de extensión Java desde su nombre de paquete.
<code>.print</code>	Muestra un mensaje en la consola del ensamblador durante la pasada final.
<code>.printnow</code>	Muestra un mensaje inmediatamente durante la evaluación; puede aparecer en varias pasadas.
<code>.pseudocommand</code>	Define una instrucción de ensamblador personalizada que expande a otras instrucciones o directivas.
<code>.pseudopc</code>	Ensambla direcciones como si el código estuviera en otro lugar; no lo copia ni lo reubica durante la ejecución.
<code>.return</code>	Devuelve un resultado desde una función de script. No es la instrucción rts.
<code>.segment</code>	Selecciona uno o más segmentos de salida según la configuración; admite parámetros del bloque.
<code>.segmentdef</code>	Define un segmento con inicio, límites y otras propiedades.
<code>.segmentout</code>	Emite los bytes de segmentos intermedios dentro del bloque actual.
<code>.struct</code>	Declara una estructura de valores del lenguaje de script; no es por sí sola una estructura en RAM.
<code>.te</code>	Alias de <code>.text</code> .
<code>.text</code>	Emite los bytes de una cadena usando la codificación actual.
<code>.var</code>	Declara una variable de ensamblado. No crea una variable que cambie durante el juego.
<code>.watch</code>	Añade una vigilancia de dirección o rango a los datos de depuración para el debugger compatible.
<code>.while</code>	Repite directivas mientras una expresión de ensamblado sea verdadera.
<code>.wo</code>	Alias de <code>.word</code> .
<code>.word</code>	Emite palabras de 16 bits: byte bajo primero, luego byte alto.
<code>.zp</code>	Marca las etiquetas de un bloque como pertenecientes a página cero, incluso antes de resolverlas.

Estas directivas se procesan al ensamblar. Las que emiten bytes preparan el archivo; no escriben por sí solas sobre una C64 que ya está ejecutando el programa.

La sintaxis completa de parámetros para segmentos, archivos y discos está en los capítulos 10 y 11 del manual de Kick Assembler.

07 / Preprocesador y utilidades

El preprocesador selecciona el código fuente antes de su evaluación.

Directiva	Para qué sirve
<code>#define</code>	Define un símbolo booleano de preprocesador. No es una constante numérica como <code>.const</code> .
<code>#undef</code>	Elimina la definición de un símbolo de preprocesador.
<code>#if</code>	Incluye un bloque si la expresión de símbolos es verdadera.
<code>#elif</code>	Prueba otra condición después de un <code>#if</code> o <code>#elif</code> que no se cumplió.
<code>#else</code>	Incluye la alternativa si ninguna condición anterior se cumplió.
<code>#endif</code>	Termina el bloque condicional.
<code>#import</code>	Incorpora otro archivo fuente.
<code>#importif</code>	Incorpora un archivo fuente solo si se cumple la condición indicada.
<code>#importonce</code>	Dentro de un archivo, evita que se procese más de una vez por importación.

Operadores del preprocesador: `!`, `&&`, `||`, `==`, `!=` y paréntesis. Sus símbolos indican definido/no definido; no son las variables numéricas del programa.

UTILIDADES DEL ENSAMBLADOR: SELECCIÓN PARA CONSULTA

Referencia	Descripción
<code>BasicUpstart</code> / <code>BasicUpstart2</code>	Macros incorporadas que generan una línea BASIC para arrancar código máquina. La segunda organiza también las posiciones de memoria.
<code>LoadBinary</code> / <code>LoadSid</code> / <code>LoadPicture</code>	Cargan datos, música SID o imágenes para procesarlos al ensamblar. Son funciones, no instrucciones de CPU.
<code>List</code> / <code>Hashtable</code> / <code>.struct</code>	Colecciones y estructuras para generar datos mediante el lenguaje de script.
<code>floor</code> / <code>ceil</code> / <code>round</code> / <code>mod</code>	Redondeo hacia abajo, arriba, al más cercano y resto de división.
<code>sin</code> / <code>cos</code> / <code>toRadians</code>	Cálculos para tablas; las funciones trigonométricas usan radianes.
<code>getFilename</code> / <code>getPath</code>	Obtienen el nombre o la ruta del archivo fuente.
<code>createFile</code>	Crea una salida adicional mediante el lenguaje de script.
<code>cmdLineVars</code>	Tabla de argumentos de script recibidos desde la línea de comandos.

Esta selección de funciones no pretende enumerar todos los métodos de la biblioteca de script. Las directivas y los operadores documentados sí están catalogados en las páginas anteriores.

08 / ROM KERNAL: ACPTR - IOINIT

Las 39 entradas públicas de la tabla KERNAL, ordenadas por nombre.

Rutina	Dirección	Para qué sirve / parámetros
ACPTR	\$FFA5	Recibe un byte del bus serie IEC y lo devuelve en A. Requiere preparar al dispositivo para transmitir.
CHKIN	\$FFC6	Selecciona como entrada el archivo lógico cuyo número está en X; debe haberse abierto antes.
CHKOUT	\$FFC9	Selecciona como salida el archivo lógico indicado en X; debe haberse abierto antes.
CHRIN	\$FFCF	Lee un carácter del canal de entrada en A. Desde teclado usa entrada de línea y puede esperar.
CHROUT	\$FFD2	Envía el byte PETSCII de A al canal de salida. En pantalla interpreta caracteres y controles.
CINT	\$FF81	Inicializa el editor de pantalla y el VIC-II. Afecta configuración de video y estado del editor.
CIOUT	\$FFA8	Envía el byte de A al bus serie; requiere LISTEN/SECOND. La salida puede quedar temporalmente almacenada.
CLALL	\$FFE7	Limpia la tabla de archivos lógicos y restaura los canales predeterminados. Para cerrar correctamente archivos de disco, usar CLOSE.
CLOSE	\$FFC3	Cierra el archivo lógico indicado en A y libera sus recursos asociados.
CLRCHN	\$FFCC	Restaura teclado como entrada y pantalla como salida. No equivale a cerrar todos los archivos.
GETIN	\$FFE4	Obtiene un byte del canal de entrada. Con teclado devuelve A=0 si no hay carácter disponible; no espera.
IOBASE	\$FFF3	Devuelve la base de E/S en X (bajo) e Y (alto). En C64: \$DC00.
IOINIT	\$FF84	Inicializa hardware de E/S, como las CIA, y estado asociado. Se usa durante el arranque.

Los nombres son convenciones de documentación: Kick Assembler no define automáticamente CHROUT, GETIN, etc. Hay que definir esos símbolos o usar la dirección.

Las llamadas ordinarias se hacen con jsr y regresan con rts. La ROM debe estar visible en el mapa de memoria; no asumir que A, X, Y o los flags se conservan salvo documentación explícita.

Las rutinas de teclado, pantalla y reloj dependen del estado del KERNAL y, en varios casos, de su IRQ. GETIN lee el buffer; no explora por sí sola la matriz del teclado.

La salida KERNAL de pantalla usa el editor de caracteres. No proporciona un motor de texto ni una rutina de borrado de bitmap.

08 / ROM KERNAL: LISTEN - SCREEN

Las 39 entradas públicas de la tabla KERNAL, ordenadas por nombre.

Rutina	Dirección	Para qué sirve / parámetros
LISTEN	\$FFB1	Ordena escuchar al dispositivo IEC cuyo número está en A.
LOAD	\$FFD5	A=0 carga; A=1 verifica. Usa SETNAM/SETLFS; X/Y indican destino si dirección secundaria=0. C=1 señala error.
MEMBOT	\$FF9C	C=1 lee el límite inferior de memoria en X/Y; C=0 lo fija desde X/Y. X=bajo, Y=alto.
MEMTOP	\$FF99	C=1 lee el límite superior de memoria en X/Y; C=0 lo fija. No reubica automáticamente un programa BASIC existente.
OPEN	\$FFC0	Abre un archivo lógico usando SETLFS y, si corresponde, SETNAM. C=1 indica error; A contiene el código.
PLOT	\$FFF0	C=0 fija cursor: X=fila, Y=columna. C=1 devuelve la posición en esos registros. Coordenadas desde 0.
RAMTAS	\$FF87	Inicializa zonas de RAM y punteros del sistema y determina límites de memoria. Rutina de arranque, no de limpieza gráfica.
RDTIM	\$FFDE	Lee el reloj de 24 bits: A=byte bajo, X=medio, Y=alto. Se incrementa nominalmente a 60 ticks por segundo.
READST	\$FFB7	Devuelve en A el estado de E/S, incluidos indicadores como fin de archivo. No confundir con el registro P del procesador.
RESTOR	\$FF8A	Restaura los vectores KERNAL de RAM a sus destinos predeterminados.
SAVE	\$FFD8	Guarda un rango. A apunta al par de página cero con el inicio; X/Y contienen el final exclusivo. Preparar SETNAM/SETLFS.
SCNKEY	\$FF9F	Explora el teclado y actualiza su buffer. Normalmente se ejecuta desde la IRQ del sistema.
SCREEN	\$FFED	Devuelve dimensiones de pantalla: X=columnas, Y=filas; normalmente 40 y 25.

Los nombres son convenciones de documentación: Kick Assembler no define automáticamente CHROUT, GETIN, etc. Hay que definir esos símbolos o usar la dirección.

Las llamadas ordinarias se hacen con jsr y regresan con rts. La ROM debe estar visible en el mapa de memoria; no asumir que A, X, Y o los flags se conservan salvo documentación explícita.

Las rutinas de teclado, pantalla y reloj dependen del estado del KERNAL y, en varios casos, de su IRQ. GETIN lee el buffer; no explora por sí sola la matriz del teclado.

La salida KERNAL de pantalla usa el editor de caracteres. No proporciona un motor de texto ni una rutina de borrado de bitmap.

08 / ROM KERNAL: SECOND - VECTOR

Las 39 entradas públicas de la tabla KERNAL, ordenadas por nombre.

Rutina	Dirección	Para qué sirve / parámetros
SECOND	\$FF93	Envía la dirección secundaria IEC después de LISTEN; A contiene el byte de comando, normalmente \$60 OR secundaria.
SETLFS	\$FFBA	Prepara A=número de archivo lógico, X=dispositivo, Y=dirección secundaria.
SETMSG	\$FF90	Controla mensajes KERNAL con A: bit 7 para mensajes de control, bit 6 para errores; 0 los suprime.
SETNAM	\$FFBD	Prepara nombre de archivo: A=longitud, X=byte bajo del puntero, Y=byte alto. No necesita terminador cero.
SETTIM	\$FFDB	Fija el reloj de 24 bits: A=byte bajo, X=medio, Y=alto.
SETTMO	\$FFA2	Entrada de compatibilidad para timeout del bus. En la ROM C64 estándar no realiza una operación útil.
STOP	\$FFE1	Comprueba STOP. Devuelve Z=1 si está pulsada; puede restaurar canales como parte del tratamiento de STOP.
TALK	\$FFB4	Ordena transmitir al dispositivo IEC cuyo número está en A.
TKSA	\$FF96	Envía dirección secundaria después de TALK; A contiene el comando, normalmente \$60 OR secundaria.
UDTIM	\$FFEA	Actualiza el reloj del sistema y el estado de STOP. Normalmente la llama la IRQ; no duplicarla sin motivo.
UNLSN	\$FFAE	Envía UNLISTEN al bus serie para terminar la fase de escucha.
UNTLK	\$FFAB	Envía UNTALK al bus serie para terminar la fase de transmisión.
VECTOR	\$FF8D	Transfiere los vectores KERNAL de RAM: C=1 los copia al buffer X/Y; C=0 los instala desde ese buffer de 32 bytes.

Los nombres son convenciones de documentación: Kick Assembler no define automáticamente CHROUT, GETIN, etc. Hay que definir esos símbolos o usar la dirección.

Las llamadas ordinarias se hacen con jsr y regresan con rts. La ROM debe estar visible en el mapa de memoria; no asumir que A, X, Y o los flags se conservan salvo documentación explícita.

Las rutinas de teclado, pantalla y reloj dependen del estado del KERNAL y, en varios casos, de su IRQ. GETIN lee el buffer; no explora por sí sola la matriz del teclado.

La salida KERNAL de pantalla usa el editor de caracteres. No proporciona un motor de texto ni una rutina de borrado de bitmap.

09 / ROM: uso práctico y entradas internas

Las direcciones internas siguientes son una selección, no una API pública ni un inventario de toda la ROM.

Entrada	Función / condición
CHROUT: A=\$93	Con salida a pantalla, limpia caracteres y lleva el cursor al inicio. No borra 8000 bytes de bitmap.
CHROUT: A=\$13	Lleva el cursor al inicio de la pantalla sin limpiarla.
CHROUT: A=\$0D	Retorno de carro mediante el editor.
CHROUT: A=\$11 / \$91	Mueve el cursor abajo / arriba.
CHROUT: A=\$1D / \$9D	Mueve el cursor derecha / izquierda.
CHROUT: A=\$12 / \$92	Activa / desactiva caracteres en reverso.
\$AB1E / STROUT	BASIC V2: imprime una cadena terminada en cero. A=byte bajo del puntero, Y=alto; usa el estado de BASIC.
\$BD CD / LINPRT	BASIC V2: imprime un entero sin signo de 16 bits en decimal. A=byte alto, X=byte bajo.
\$BDDD / FOUT	BASIC V2: convierte el acumulador flotante FAC a una cadena; devuelve el puntero en A (bajo) e Y (alto). Usa un buffer en la página de pila.
\$E544	Editor: limpia la pantalla de caracteres y restablece su estado de líneas/cursor. Es una dirección interna; preferir CHROUT con \$93.
\$E566	Editor: lleva el cursor al inicio. Dirección interna; preferir CHROUT con \$13 o PLOT.

El KERNAL ocupa normalmente \$E000-\$FFFF; BASIC V2, \$A000-\$BFFF. Hay además código auxiliar de BASIC en la zona alta. No toda dirección dentro de una ROM es una entrada válida a una subrutina.

Las rutinas internas pueden depender de variables en página cero, FAC, punteros y pila. Sus nombres varían entre listados. Para otras entradas, consultá el desensamblado enlazado en Fuentes.

CONTRATOS QUE CONVIENE RECORDAR

Referencia	Descripción
Archivos	SETNAM y SETLFS preparan parámetros; OPEN abre; CHKIN/CHKOUT seleccionan canal; CHRIN/CHROUT transfieren; CLRCHN restaura; CLOSE cierra.
Error de E/S	Comprobar C y A en las llamadas que documentan error por carry; READST obtiene otros bits de estado del canal. No todas las rutinas usan el mismo contrato.
PLOT vs SCREEN	PLOT usa X=fila e Y=columna. SCREEN devuelve X=ancho e Y=alto. No intercambiarlos.

10 / Memoria y video: recordatorio

Direcciones útiles para este proyecto. La configuración del banco de memoria puede cambiar su interpretación.

Dirección / dato	Referencia rápida
\$0000 / \$0001	Dirección y datos del puerto del 6510; participan en la selección de RAM, ROM y E/S.
\$0100-\$01FF	Pila del procesador: 256 bytes. jsr, interrupciones y operaciones push/pull la utilizan.
\$0400-\$07E7	1000 celdas de pantalla en la configuración inicial. Caracteres en modo texto; colores en bitmap.
\$07F8-\$07FF	Punteros de sprites con pantalla en \$0400. No son celdas visibles. Cambian al mover la memoria de pantalla.
\$D000-\$D00F	Coordenadas X bajas e Y de los ocho sprites, intercaladas.
\$D010	Bit alto de X de cada sprite.
\$D011	Control de video: bit 5 activa bitmap; bit 6 selecciona modo extendido. Bit 7 tiene distinta función al leer y escribir.
\$D012	Lectura: línea raster baja. Escritura: comparador de IRQ raster bajo.
\$D015	Habilita sprites: un bit por sprite.
\$D016	Control horizontal y multicolor: bit 4 habilita multicolor.
\$D017 / \$D01D	Expansión vertical / horizontal de sprites.
\$D018	Selecciona memoria de pantalla y fuente o bitmap dentro del banco VIC-II.
\$D019 / \$D01A	Estado/confirmación de interrupciones VIC-II y máscara de habilitación. En \$D019 se escriben unos para reconocer eventos.
\$D01B / \$D01C	Prioridad respecto del fondo / multicolor de sprites.
\$D01E / \$D01F	Colisiones sprite-sprite / sprite-fondo. La lectura limpia los indicadores acumulados.
\$D020 / \$D021	Color del borde / primer color de fondo; usan los cuatro bits bajos.
\$D025-\$D026	Dos colores compartidos por sprites multicolor.
\$D027-\$D02E	Color individual de cada sprite.
\$D800-\$DBE7	RAM de color para las 1000 celdas; cuatro bits útiles por posición.
\$DC00 / \$DC01	Puertos CIA1: teclado y joysticks. Para leer matriz hay que configurar correctamente sus direcciones.
\$DD00	Los dos bits bajos del puerto CIA2 seleccionan el banco VIC-II de 16 KiB, con codificación invertida.
Bitmap / sprite	Bitmap: 8000 bytes en un bloque alineado a 8 KiB. Sprite: 63 bytes de dibujo en un bloque alineado a 64 bytes.

Bitmap hires: 320 x 200; cada bloque de 8 x 8 elige dos colores mediante los dos nibbles de su byte de pantalla. Bitmap multicolor: 160 x 200; cada par de bits elige entre cuatro colores.

Precaución de lectura-modificación-escritura: \$D011 leído devuelve el bit alto del raster actual, pero al escribir programa el comparador de IRQ. No todos los registros de hardware son RAM ordinaria.

11 / Fuentes y alcance

Referencia para C64 clásica y el ensamblador instalado en este proyecto.

BASE DOCUMENTAL

Kick Assembler Reference Manual, Mads Nielsen

<https://theweb.dk/KickAssembler/>

Base principal: KickAssembler.pdf de tu proyecto. Capítulos 3-9, 10-14, 16-17 y apéndice A. Se cotejó el catálogo local, incluida .watch, que no figura en su tabla resumida.

Manual HTML de Kick Assembler

<https://theweb.dk/KickAssembler/webhelp/content/index.html>

Para consultar parámetros completos, ejemplos de directivas y la biblioteca de funciones de script.

Commodore 64 Programmer's Reference Guide: KERNAL

https://www.devili.iki.fi/Computers/Commodore/C64/Programmers_Reference/Chapter_5/page_272.html

Tabla original de las 39 entradas públicas. Las direcciones de esta guía corresponden a C64, no a C128 ni a otros modelos.

KERNAL API: documentación y fuentes contrastadas

<https://www.pagetable.com/c64ref/kernal/>

Incluye la documentación original de Commodore, contratos y comentarios sobre las diferencias entre rutinas.

ROM de la C64: listado y fuentes originales

<https://www.pagetable.com/c64ref/c64disasm/>

Para verificar entradas internas, registros de entrada, variables temporales y el flujo completo de BASIC/KERNAL.

6502 Family CPU Reference

<https://www.pagetable.com/c64ref/6502/>

Consulta interactiva para bytes de opcode, ciclos y variantes. Seleccionar NMOS 6502/6510, no 65C02.

Mapa de memoria C64

<https://www.pagetable.com/c64ref/c64mem/>

Ampliación de direcciones de RAM, ROM, VIC-II, SID y CIA.

CÓMO USAR ESTA GUÍA

Los parámetros de ROM son un resumen operativo, no una lista completa de registros destruidos y efectos laterales. Para una llamada nueva, contrastá su contrato completo. Los modos de direccionamiento aparecen junto a cada mnemónico; el costo en ciclos depende del modo y, a veces, de cruces de página.

Las descripciones están redactadas para consulta rápida. Las extensiones exclusivas del 65C02/DTV y el inventario completo de métodos de script quedan fuera del alcance C64 de esta edición.